Introducción a Git y a Github

Diego Sevilla Ruiz

DITEC
Facultad de Informática

Murcia, 2026

Índice

1. Introducción a Git
2. Github

Índice

1 Introducción a Git

2 Github

Introducción a Git y Github

- Git es un sistema de control de versiones distribuido
- Esto significa que no hay una copia "maestra" del repositorio, sino que cada copia local contiene toda la información (si está actualizada)
- Fue creado inicialmente por Linus Torvalds cuando el gestor de versiones que usaba para el núcleo de Linux dejaron de ofrecerlo gratuitamente
- Permite que un conjunto de personas trabajen de forma local en un repositorio compartido, en el que periódicamente van enviando sus cambios solucionando los posibles conflictos

Conceptos

Repositorio Un repositorio en Git incluye los ficheros de la copia de trabajo bajo control de versiones junto con todo el árbol de *commits*, es decir, cada uno de los cambios realizados sobre los ficheros. Si el repositorio es *bare*, no contiene los ficheros en sí, sólo el árbol (que, a su vez, al incluir todos los *commits*, permite reconstruir los ficheros)

Commit Un *commit* agrupa cambios sobre uno o varios ficheros de la copia de trabajo. Cada *commit* está identificado por un *hash* SHA-1

Versión/Árbol de versiones En Git, una *versión* no es más que un punto en el árbol (un *commit*). Guarda cómo estaban los ficheros en ese momento. Un *commit* se realiza a partir de uno o varios *commits* anteriores. Así se forma el árbol de *commits*. La raíz del árbol se denomina **HEAD**

Ramas Una rama se representa en el árbol como una bifurcación partiendo de un *commit*. Una rama, al igual que una versión, no es más que un puntero al *commit* más avanzado de esa bifurcación

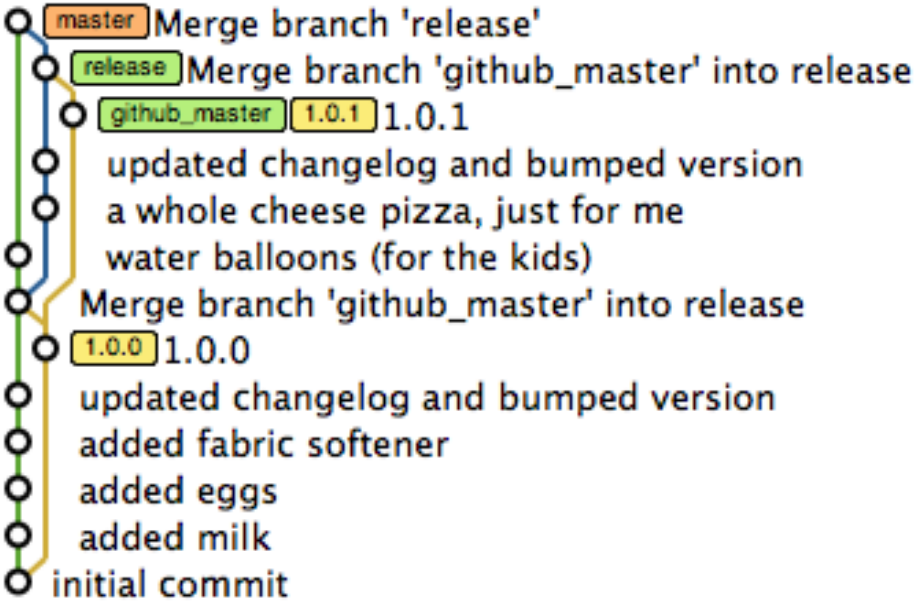
Conceptos II

Zona *Staging* Git también mantiene un área llamada "*staging*" (en español algo así como "en preparación"). Este área mantiene los ficheros modificados temporalmente pero que todavía no han ido a parar a un *commit*

Repositorios locales/Remotos Git es distribuido, por lo que cada repositorio, nuestro repositorio local y los posibles remotos funcionan como **pares**. Opcionalmente se puede configurar un repositorio como el "origen", contra el que se puede sincronizar por defecto, pero se permiten otras configuraciones (p. ej. varios remotos, o que nuestro repositorio local haga de remoto de otro, etc.)

Push/Pull El envío y la recepción de nuevas versiones y nuevos *commits* está separado de los *commits* en sí, y se realiza con instrucciones *push*/*pull*.

Árbol de commits



Clone

- La forma más sencilla de crear un repositorio Git local es ***clonando*** un repositorio existente
- Al ser un sistema distribuido, cada "clon" de un repositorio es idéntico y guarda toda la información hasta ese momento
- Los clones se pueden realizar a través de diferentes protocolos, local, http, ssh, etc.
- La sintaxis de git para clonar es la siguiente:

```
$ git clone URL
```

  donde `URL` es la URL de clon del repositorio (tanto en Gitlab como en Github se puede obtener de cualquier repositorio)
- Esto crea un directorio con el nombre del repositorio, y una copia del mismo

Añadiendo nuevo contenido

- Una vez se tiene un repositorio, se puede añadir los cambios que hagamos al *staging*
- Los ficheros y modificaciones en *staging* se mantienen mientras se están modificando ficheros o añadiendo ficheros nuevos
- Los cambios en *staging* se pueden desechar (*stash*) o se pueden fijar haciendo *commit*
- Las modificaciones sobre los ficheros **no pasan automáticamente al área de *staging***
- Al añadir o modificar un objeto, para hacerlo pasar al área de *staging* hay que usar:

```
$ git add fichero
```

Añadiendo nuevo contenido II

- Al hacerlo, podemos comprobar el estado:

```
$ vi q
$ git add q
$ git status
En la rama master
Tu rama está actualizada con 'origin/master'.

Cambios a ser confirmados:
  (usa "git reset HEAD <archivo>..." para sacar
      del área de stage)

        nuevo archivo:  q
```

Confirmado (*commit*) de nuevo contenido

- Una vez añadidos los ficheros nuevos Y TAMBIÉN LOS MODIFICADOS con `git add`, podemos confirmar los cambios
- La confirmación se realiza con `git commit`:

```
$ git commit
```

- Opcionalmente se puede añadir el *flag* `-m mensaje` para añadir un mensaje al *commit*
- También está el *flag* `-a`, que añade automáticamente al *commit* todos los ficheros modificados desde el último *commit*
- Si por el contrario se quiere desechar todos los cambios desde el último *commit* se puede usar `stash`:

```
$ git stash
```

- Para ver las diferencias con el último *commit* podemos usar `git diff`
- También se pueden eliminar ficheros con `git rm` y mover ficheros con `git mv`

Enviando los nuevos commits al repositorio remoto

¡Cuidado!

El *commit* sólo se hace en nuestro repositorio local

Push

Para enviar los *commits* al repositorio remoto se utiliza `git push`

Versiones e identificadores, *log*

- Cada *commit* tiene asociado un número único *hash* SHA-1
- Además, cada *commit* se enlaza con el anterior (o los anteriores) en el tiempo (árbol)

```
commit 580513593c6913408e8775c9f6139bd7424d37ad (
    HEAD -> master, origin/master, origin/HEAD)
Author: Diego Sevilla Ruiz <dsevilla@ditec.um.es>
Date:   Mon Feb 12 17:45:55 2018 +0100

    mensaje

commit db80ae2fdf352d390f82c7bd368c09539fc2bf32
Author: Diego Sevilla Ruiz <dsevilla@ditec.um.es>
Date:   Mon Feb 12 01:20:24 2018 +0100

    mensaje
```

- Hay opciones interesantes como `--stat` y `--graph`, que muestran las ramas (si existen) y cómo se han mezclado (más de esto después)

Recibiendo cambios

Dos situaciones:

Si no tenemos cambios locales

Si no tenemos cambios locales nuevos, lo más sencillo es hacer:

```
$ git pull
```

Esto trae todos los cambios y actualiza nuestro directorio de trabajo

Recibiendo cambios II

Si tenemos commits locales sin enviar al servidor

Si tenemos cambios nuevos y *commits* locales sin enviar al servidor, podemos obtener los cambios del servidor con la construcción:

```
$ git fetch
```

Esto NO cambia nuestro directorio de trabajo. Actualiza el árbol local para incluir los commits remotos, pero NO nuestro directorio de trabajo. Podemos seguir haciendo *commits*.
Para **mezclar los cambios hechos en el servidor con los nuestros**, podemos utilizar:

```
$ git rebase
```

Esto coloca nuestros cambios **ENCIMA** de los últimos cambios traídos. Todavía tendremos que hacer `git push` para enviarlos

Conflictos

- Lo usual es que los cambios se mezclen sin problema
- Sin embargo, el paso `git rebase` puede dejar los ficheros en un estado inconsistente si **dos personas han modificado el mismo fichero en el mismo sitio**
- Por ejemplo, en un fichero `t`, se ha producido un conflicto:

```
$ git rebase
Primero, rebobinando head para reproducir tu trabajo en
    la cima de él...
Aplicando: t
Usando la información del índice para reconstruir un á
    rbol base...
M       t
Retrocediento para parchear base y fusión de 3-vías...
Auto-fusionando t
CONFLICTO (contenido): Conflicto de fusión en t
error: Falló al fusionar en los cambios.
El parche fallo en 0001 t
La copia del parche que fallido se encuentra en: .git/
    rebase-apply/patch
```

Conflictos II

```
Resuelva todos los conflictos manualmente ya sea con
"git add/rm <archivo_conflictivo>", luego ejecute "git
    rebase --continue".
Si prefiere saltar este parche, ejecute "git rebase --
    skip" .
Para abortar y regresar al estado previo al "git rebase"
    , ejecute "git rebase --abort".
```

Conflictos III

- El contenido de `t` en este momento es:

```
<<<<<<< HEAD
def
=======
ghi
>>>>>>> t
```

- La primera parte antes de "=====" representa la versión en la cabeza de los cambios (la versión remota, más de esto luego)
- La parte de abajo muestra la versión local con el último *commit*, en este caso etiquetado "`t`"
- El usuario entonces decide cómo dejar el fichero. Por ejemplo, puede dejar ambas líneas una detrás de otra en el fichero `t`:

```
def
ghi
```

Conflictos IV

- A continuación debe añadirlo a *staging* con:

```
$ git add t
```

- Y continuar con el *rebase* que quedó a medio:

```
$ git rebase --continue
Aplicando t
```

- Ahora se puede hacer *commit* y *push* como normalmente:

```
$ git commit -a -m "Mezclados cambios."
$ git push
```

Ramas

- Las ramas en Git permiten el desarrollo paralelo con respecto a la rama *master* remota
- Nuestra rama *master* local se sincroniza por defecto con la rama *master* remota, y su posición es siempre "HEAD" (cabeza del árbol)
- Los desarrollos paralelos en ramas permiten tener varias versiones del desarrollo, o trabajar temporalmente en modificaciones y pruebas que pueden incorporarse a la rama raíz (piénsense como ramas reales del árbol de versiones)
- El proceso de unión de cambios de una rama en otra se llama mezcla (*merge*)
- En Git, tanto la posición HEAD como cada rama, como también cada *tag* (los *tags* se utilizan para señalar puntos específicos del desarrollo) son simplemente **apuntadores a un SHA-1 de un commit**, es decir, **punteros al árbol donde están relacionados todos los commits**

Ramas II

- Las ramas se pueden mantener sólo localmente o bien se pueden sincronizar con el servidor
- Se pueden ver las ramas existentes con

```
$ git branch -a
* master
  remotes/origin/HEAD -> origin/master
  remotes/origin/master
```

La salida muestra las ramas locales (aquí sólo `master`), qué rama está siendo usada ahora (la que lleva el asterisco) y las ramas en el repositorio remoto (llamado por defecto `origin` por git). En este caso la versión `HEAD` corresponde con la posición de `master` en el repositorio remoto

Ramas III

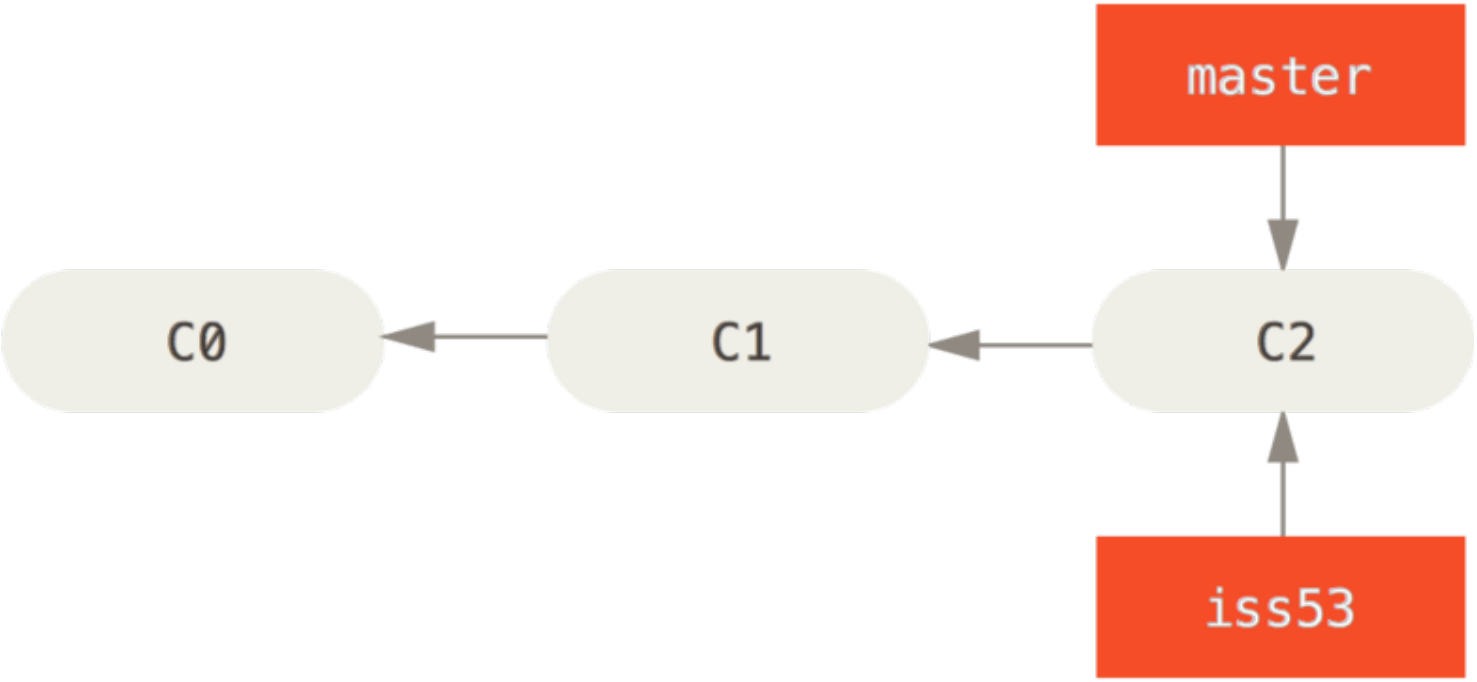
- Para obtener un estado concreto del código en un *commit* ("volver" a esa versión o rama) se puede usar:

```
$ git checkout <SHA-1>
```

- Nuestra copia de trabajo queda exactamente como estaba tras ese *commit*
- Si añadimos el parámetro `-b` y el nombre de una rama, creamos un apuntador a la posición actual con el nombre que le demos:

```
$ git checkout -b iss53
```

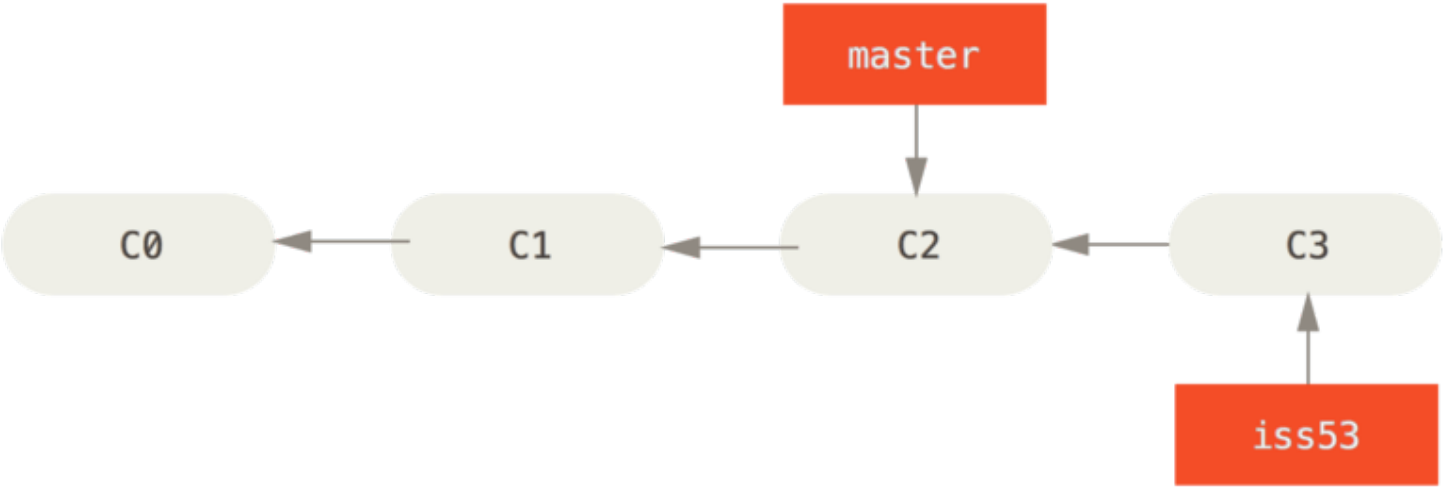
Ramas IV



- A partir de ese momento estamos en la rama `iss53`

Ramas V

- Si hacemos un commit, la situación será:



- Si queremos volver a la rama `master`:

```
$ git checkout master
```

- Si después hacemos un nuevo *commit*, `C4`, éste tendrá como padre a `C2`, y ambas ramas, `master` y `iss53` seguirán en paralelo

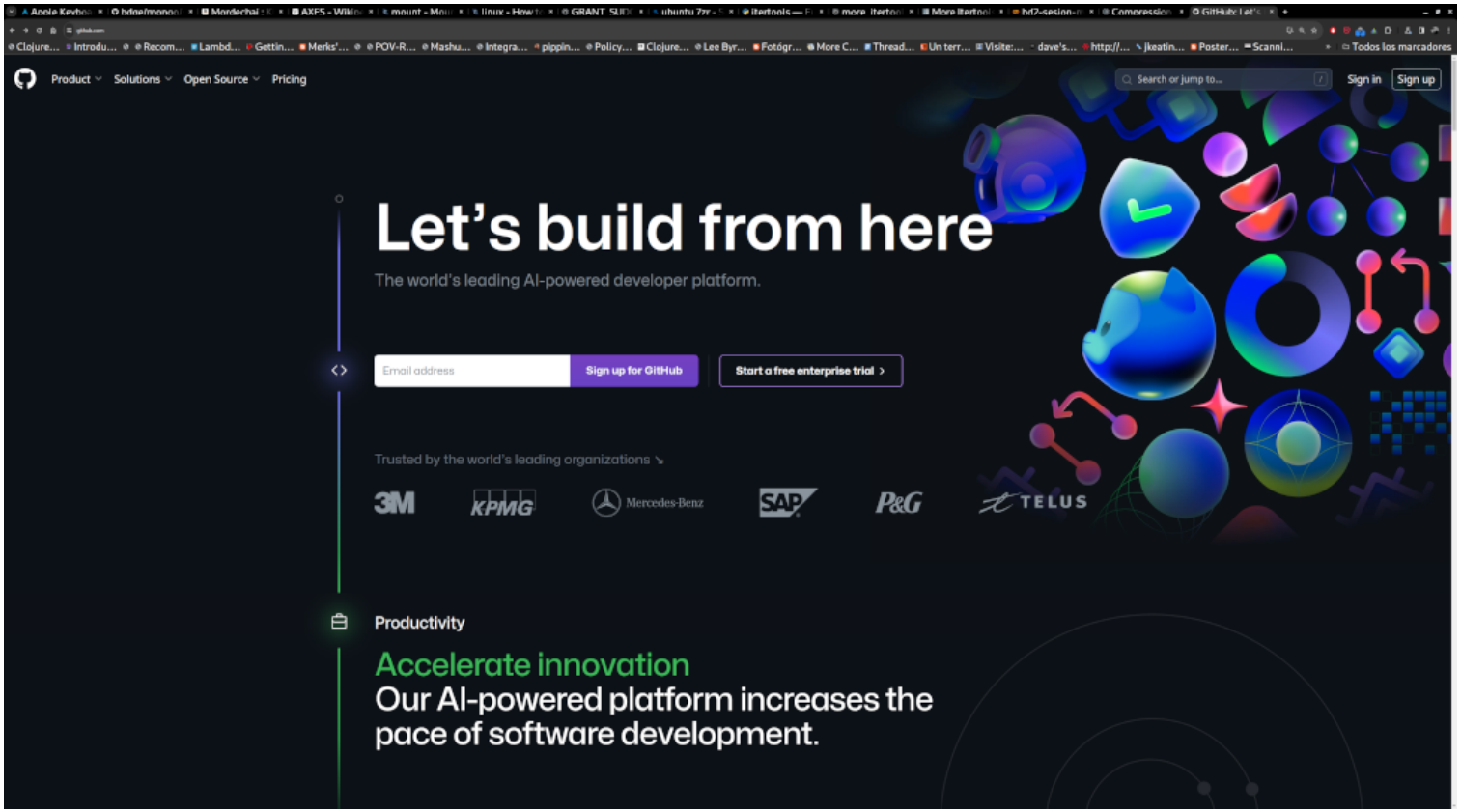
Índice

1 Introducción a Git

2 Github

Introducción a Github

- Github es un interfaz Web para gestionar repositorios Git y usuarios
- En `https://github.com/` se puede acceder a la aplicación. Hay que registrarse como usuario



Creación de proyecto

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

Owner * Repository name *

dsevilla /

Great repository names are short and memorable. Need inspiration? How about special-tribble ?

Description (optional)

Public
Anyone on the internet can see this repository. You choose who can commit.

Private
You choose who can see and commit to this repository.

Initialize this repository with:

Add a README file
This is where you can write a long description for your project. Learn more about READMEs.

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. Learn more about ignoring files.

Choose a license

License: None

A license tells others what they can and can't do with your code. Learn more about licenses.

You are creating a public repository in your personal account.

Create repository

Página del proyecto test1

ssdd-grupos-2324 Public

Pin | Unwatch 1 | Fork 0 | Star 0

main | 1 Branch | 0 Tags | Go to file t | Add file | Code

dsevilla Initial commit 30fa9d4 · 1 minute ago 1 Commits

.gitignore	Initial commit	1 minute ago
LICENSE	Initial commit	1 minute ago
README.md	Initial commit	1 minute ago

README | MIT license

ssdd-grupos-2324

Descripción de los grupos y repositorios de la asignatura.

About

Descripción de los grupos y repositorios de la asignatura.

Readme
Activity
0 stars
1 watching
0 forks

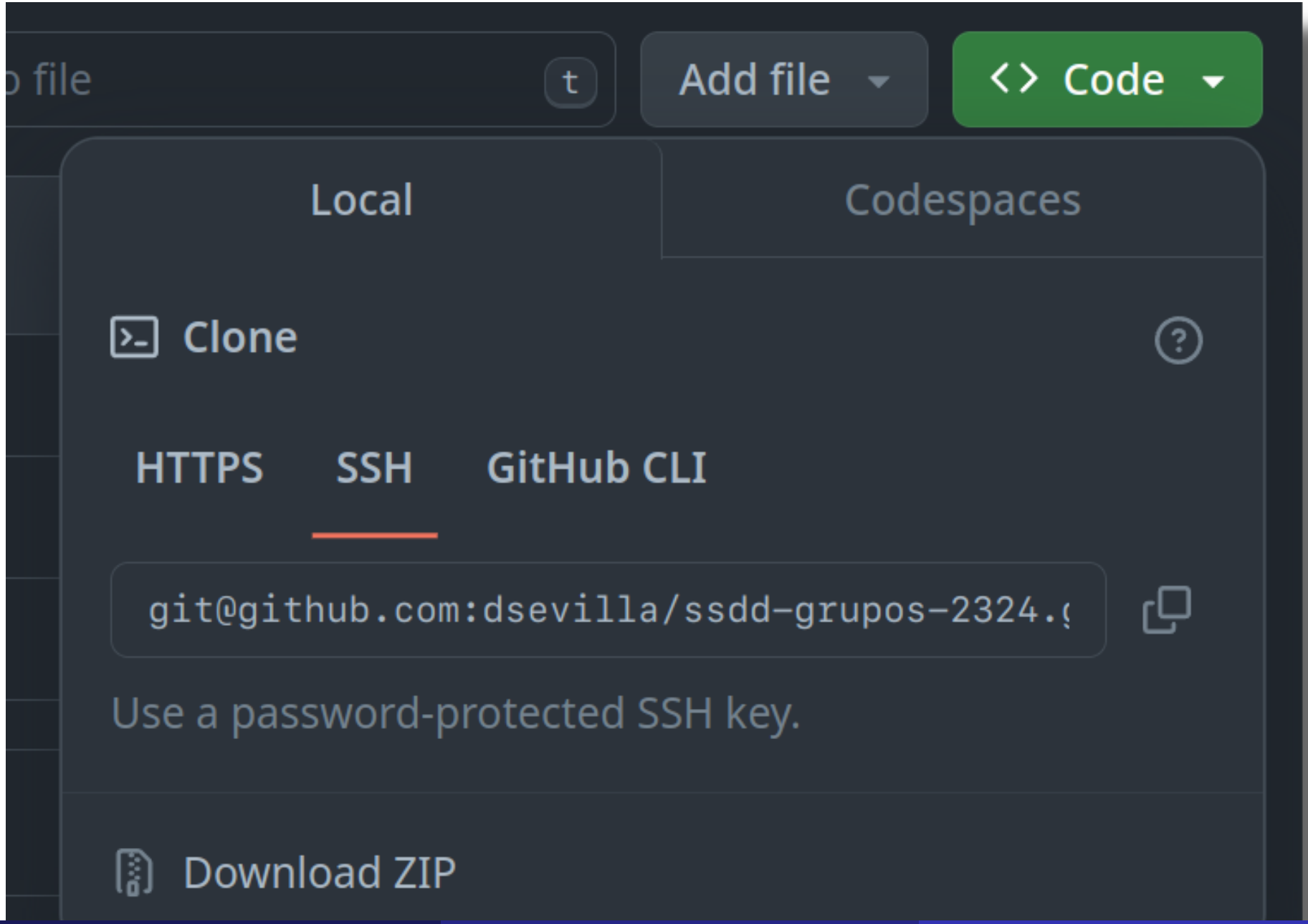
Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Página del proyecto test1 (instrucciones)



Proyecto

- La URL de clonado del proyecto aparece ahí
- Se puede descargar en modo sólo lectura usando la URL `https://...`
- El *combobox* permite seleccionar una URL basada en el protocolo SSH
- Esto permite tener un repositorio de lectura y escritura, pero **requiere de la configuración de las claves SSH**

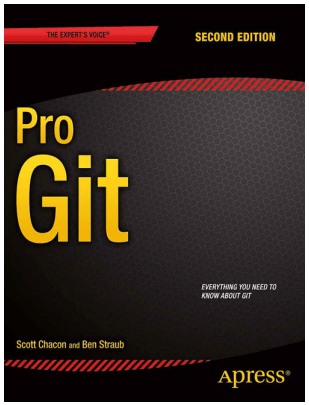
Claves SSH

- Las claves SSH se pueden generar para cada usuario y ordenador si no están ya creadas
- Residen en el directorio *home* del usuario, en el subdirectorio "`.ssh`"
- Las claves pueden ser RSA o DSA, que corresponden a los ficheros `id_rsa` y `id_rsa.pub` o `id_dsa` y `id_dsa.pub`
- Los ficheros que nos interesan son las claves públicas (archivos `.pub`)
- El contenido de los mismos se puede copiar en la página del proyecto
- Si no existen los ficheros, se pueden generar con OpenSSL:

```
$ ssh-keygen -t rsa -b 4096 -C "email@example.com"
```

(pide una clave, que se suele dejar vacía)

Referencias



- https://git-scm.com/book/en/v2 (también en español)
- https://github.com/progit/progit2/releases/download/2.1.38/progit.pdf